

Kubernetes 3-Node Cluster Setup

k8s-master (Control Plane) + k8s-worker1 + k8s-worker2

Practical setup guide based on the commands used during the lab.

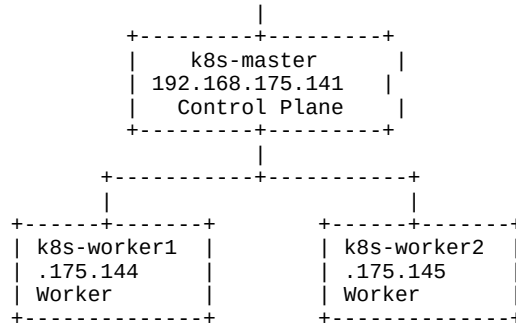
Node	Hostname	IP	Role
Master	k8s-master	192.168.175.141	Control Plane
Worker 1	k8s-worker1	192.168.175.144	Worker
Worker 2	k8s-worker2	192.168.175.145	Worker

Environment: Ubuntu 24.04.4 LTS • Kubernetes v1.36.3 • containerd 2.2.1 • Flannel • VMware Workstation

Security note: The original kubeadm join token was intentionally not printed in this PDF. It was exposed during the lab; generate a fresh join command if it is ever needed again.

1. Final Architecture

Kubernetes Cluster



Pod CIDR: 10.244.0.0/16
CNI: Flannel
Runtime: containerd

2. Common Commands — RUN ON ALL THREE NODES

These preparation commands were performed on k8s-master, k8s-worker1 and k8s-worker2.

2.1 Hostname and IP verification

```
hostname
hostname -I
```

Hostnames were set individually:

```
sudo hostnamectl set-hostname k8s-master
sudo hostnamectl set-hostname k8s-worker1
sudo hostnamectl set-hostname k8s-worker2
```

2.2 Network connectivity test

```
# From master
ping -c 3 192.168.175.144
ping -c 3 192.168.175.145
```

The earlier 192.168.175.14 test failed because the address was mistyped; .144 and .145 worked.

2.3 Install prerequisites

```
sudo apt update
sudo apt install -y apt-transport-https ca-certificates curl gpg
sudo mkdir -p -m 755 /etc/apt/keyrings
```

2.4 Install containerd

```
sudo apt update
sudo apt install -y containerd
containerd --version
```

2.5 Configure containerd

```
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml
sudo nano /etc/containerd/config.toml
```

Inside the file, change:

```
SystemdCgroup = false
```

to:

```
SystemdCgroup = true
sudo systemctl restart containerd
sudo systemctl enable containerd
sudo systemctl status containerd
sudo systemctl is-active containerd
```

2.6 Kubernetes repository — v1.36

The old v1.28 repository was removed and replaced with v1.36 so all nodes use the same release line.

```
sudo rm -f /etc/apt/sources.list.d/kubernetes.list

curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.36/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.36/deb/ ' | sudo tee /etc/apt/sources.list.d/kubernetes.list

sudo apt update
apt-cache policy kubeadm
```

The lab showed kubeadm candidate version 1.36.3-1.1.

2.7 Disable swap

```
sudo swapoff -a
free -h
sudo sed -i '/\sswap\s/ s/^#/' /etc/fstab
free -h
```

2.8 Install Kubernetes components

```
sudo apt update
sudo apt install -y kubelet kubeadm kubectcl
sudo apt-mark hold kubelet kubeadm kubectcl

kubeadm version
kubectcl version --client
kubelet --version
```

2.9 Kernel modules

```
sudo modprobe overlay
sudo modprobe br_netfilter
lsmod | grep -E 'overlay|br_netfilter'

cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
```

2.10 Linux networking parameters

```
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
EOF

sudo sysctl --system
sysctl net.ipv4.ip_forward
```

2.11 Enable kubelet

```
sudo systemctl enable --now kubelet
sudo systemctl status kubelet
```

Before kubeadm init/join, kubelet may show auto-restart because it has not yet received the final cluster configuration. This was observed during the lab.

3. CONTROL PLANE — k8s-master

IP: 192.168.175.141

3.1 Initialize the cluster

```
sudo kubeadm init --apiserver-advertise-address=192.168.175.141 --pod-network-cidr=10.244.0.0/16
```

This created the control-plane certificates, kubeconfig files, etcd, API server, controller manager, scheduler, bootstrap configuration, CoreDNS and kube-proxy.

3.2 Configure kubectl

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

3.3 Verify nodes

```
kubectl get nodes
```

The master initially showed NotReady because the Pod network had not yet been installed.

3.4 Install Flannel CNI

```
kubectl apply -f https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml
```

3.5 Verify cluster

```
kubectl get pods -A
kubectl get nodes
kubectl get pods -A -o wide
kubectl get nodes -o wide
```

3.6 Flannel troubleshooting commands used during startup

```
kubectl describe pod -n kube-flannel kube-flannel-ds-2xmg6
kubectl logs -n kube-flannel kube-flannel-ds-2xmg6 -c install-cni-plugin
kubectl logs -n kube-flannel kube-flannel-ds-2xmg6 -c install-cni
```

4. WORKER 1 — k8s-worker1

IP: 192.168.175.144

Worker1 received all commands from Section 2, then joined the control plane.

4.1 Verify worker identity

```
hostname  
hostname -I
```

4.2 Join the cluster

```
sudo kubeadm join 192.168.175.141:6443 \  
  --token <BOOTSTRAP-TOKEN> \  
  --discovery-token-ca-cert-hash sha256:<CA-HASH>
```

4.3 Verify local components

```
sudo systemctl status containerd  
sudo systemctl status kubelet  
kubeadm version  
kubelet --version
```

4.4 Verification was ultimately performed from the master:

```
kubectl get nodes -o wide  
kubectl get pods -A -o wide
```

5. WORKER 2 — k8s-worker2

IP: 192.168.175.145

Worker2 received all commands from Section 2, then joined the control plane.

5.1 Verify worker identity

```
hostname  
hostname -I
```

5.2 Join the cluster

```
sudo kubeadm join 192.168.175.141:6443 \  
  --token <BOOTSTRAP-TOKEN> \  
  --discovery-token-ca-cert-hash sha256:<CA-HASH>
```

5.3 Verify local components

```
sudo systemctl status containerd  
sudo systemctl status kubelet  
kubeadm version  
kubelet --version
```

5.4 Verification was ultimately performed from the master:

```
kubectl get nodes -o wide  
kubectl get pods -A -o wide
```

6. Final Verification

Run these from k8s-master after both workers have joined:

```
kubectl get nodes
kubectl get nodes -o wide
kubectl get pods -A
kubectl get pods -A -o wide
```

Final state achieved in the lab:

NAME	STATUS	ROLES	VERSION
k8s-master	Ready	control-plane	v1.36.3
k8s-worker1	Ready	<none>	v1.36.3
k8s-worker2	Ready	<none>	v1.36.3

7. Quick Command-to-Purpose Reference

Command	Purpose
containerd	Container runtime used by the Kubernetes nodes.
kubeadm init	Creates and initializes the control plane.
kubeadm join	Joins a worker node to the cluster.
kubelet	Node agent that manages Pods.
kubectl	CLI used to control/query the cluster.
kubectl get nodes	Shows node readiness and roles.
kubectl get pods -A	Shows Pods in all namespaces.
kubectl get pods -A -o wide	Shows Pod IPs and node placement.
Flannel	CNI network used for Pod-to-Pod networking.

Result: A working 3-node Kubernetes v1.36.3 cluster with one control-plane node and two worker nodes, using containerd and Flannel.

Next practical topics: Deployment → ReplicaSet → Pods → Service → scaling → self-healing → ConfigMap/Secret → volumes → Ingress.